MiniOS for a Study Room

Phase 2 progress report
Operating Systems Project | 4 November 2026

Member	KFUPM ID
Yasmeen Alshehri	202271660
Fatimah Alshehab	202278660
Raneem Alshahrani	202277080
Sarah Almas	202261060

Project recap

MiniOS connects familiar study-room activities to operating-system concepts. The C11 terminal program models one CPU, up to 20 task records, a configurable round-robin quantum, task states, a 64 KiB task-payload budget, persistent notes and simulated study sessions.

Implemented for Phase 2

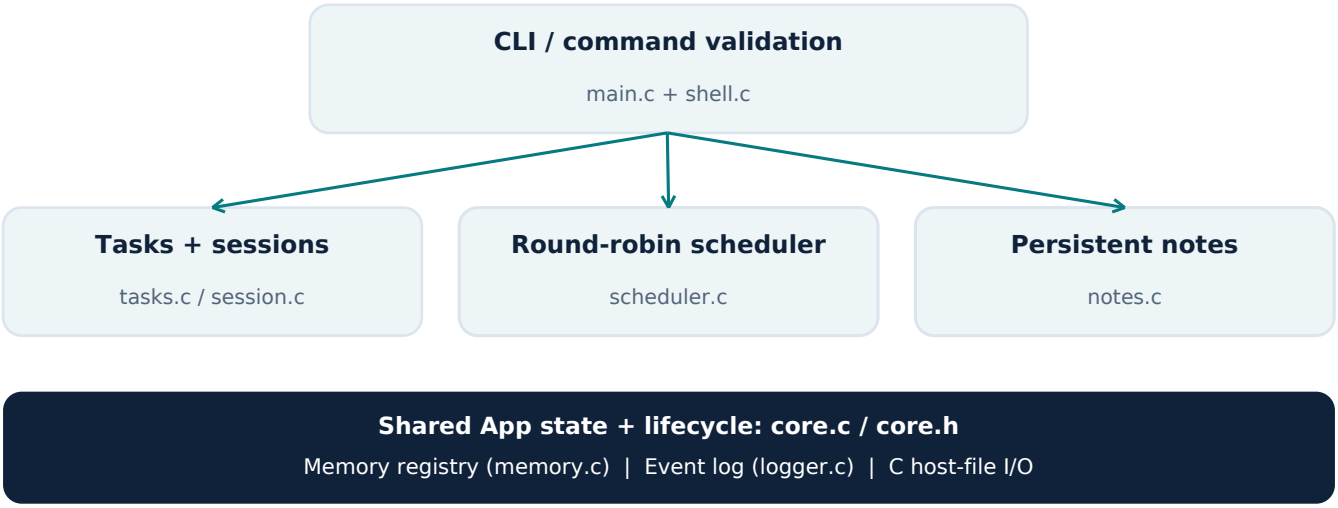
Core area	Working behavior
Tasks and CLI	Create and inspect tasks; validate commands; block/wake; safe reset and exit.
CPU scheduling	Circular FIFO ready queue; single-step and full run; virtual clock; exact reference metrics.
Memory	PID-owned malloc/free registry; 65,536-byte total limit; rollback and cleanup; failure hook.
Notes	Create, read, append and delete text notes; reopen them after a process restart.
Study sessions	Scheduled task duration with one completion notice at the finishing tick.
Evidence and export	Virtual-timestamped events, dispatch CSV, metric CSV, console captures and T1-T12 results.

Current status: the proposed essential core services are implemented and runnable. All 12 defined integration cases and the direct C assertions pass. Team-device validation, member cross-review, full leak-tool checking and final-phase presentation work remain.

Basis: the uploaded team proposal and the Phase 2 submission announcement. The announcement requires C core code, a PDF progress report and organized source as a ZIP or GitHub repository. Group submission deadline: 10 November 2026.

Architecture and design decisions

The shell validates complete commands and routes requests to the service modules. All modules share the App model. Core lifecycle functions initialize, reset, verify and clean up that model.



Change from the proposal: module responsibilities are retained. A small core.c/core.h pair centralizes shared types and lifecycle operations. The ready queue belongs to scheduler.c; test/evidence scripts remain separate from the C application.

State transitions

Trigger	Transition	Observable effect
Create	new record to READY	Allocate payload, then commit PID and queue entry.
Dispatch	READY to RUNNING	Remove the head of the FIFO ready queue.
Time slice ends	RUNNING to READY	Append unfinished work to the queue tail.
Burst completes	RUNNING to FINISHED	Record completion and free the owned payload.
Block / wake	READY to WAITING / WAITING to READY	Remove from queue / append once at tail.

Shared structures and accounting

Structure	Key contents
Task	PID, name, state, burst/remaining, arrival/first/completion times, READY waiting, payload and session flags.
ReadyQueue	Bounded circular array of PIDs, head and count. Only READY tasks are present.
AllocationRecord	Pointer, owner PID and byte count. Registry totals track live payloads.
App / Logger	Task table, queue, clock, quantum, memory registry, files and detected I/O failures.

One virtual CPU executes commands sequentially between slices. Time is virtual ticks. Context-switch overhead is modeled as zero. Notes use synchronous host-file operations; manual WAITING illustrates task states.

Measured scheduling results

The proposal's reference experiment uses bursts 4, 3 and 2, arrival time 0 for all tasks, and quantum 2. The executed trace matches the expected five slices exactly.

Round robin and measured metrics | actual C output

```
minios> run
[0,2) P1 Study        RUNNING -> READY    remaining=2
[2,4) P2 Review       RUNNING -> READY    remaining=1
[4,6) P3 Practice     RUNNING -> FINISHED remaining=0
[6,8) P1 Study        RUNNING -> FINISHED remaining=0
SESSION COMPLETE: P1 Study at tick 8
[8,9) P2 Review       RUNNING -> FINISHED remaining=0
Run paused: ready queue empty; WAITING tasks need wake.
minios> metrics
PID NAME            ARR BURST FIRST DONE TURN WAIT RESP
1   Study           0   4     0     8    8    4    0
2   Review          0   3     2     9    9    6    2
3   Practice        0   2     4     6    6    4    4
Means (3 finished): turnaround=7.67 waiting=4.67 response=2.00 ticks
minios> memory
Memory: used=0/65536 bytes free=65536 allocations=0
```

Figure 1. Actual C program stdout from tests/reference.txt. Console images in this report are faithful renderings of captured output; original text, logs and CSV files are included in evidence/.

Task	Burst	First	Done	Turnaround	Waiting	Response
P1 Study	4	0	8	8	4	0
P2 Review	3	2	9	9	6	2
P3 Practice	2	4	6	6	4	4

How the metrics are calculated

Turnaround = completion - arrival. P1: 8 - 0 = 8 ticks.
Response = first run - arrival. P2: 2 - 0 = 2 ticks.
Waiting = accumulated READY ticks. Without blocking, P1 waiting also equals 8 - 4 = 4 ticks. Blocked intervals are excluded from READY waiting.

Measured means: turnaround = (8 + 9 + 6) / 3 = **7.67**; waiting = (4 + 6 + 4) / 3 = **4.67**; response = (0 + 2 + 4) / 3 = **2.00** ticks. All payloads are released by tick 9.

Additional evidence: a separate five-task workload completes successfully in evidence/five_tasks_console.txt. Late arrivals, wrapped queues and READY-only waiting during blocking are checked by tests/test_core.c.

State and resource evidence

Blocking removes P1 from the ready queue. P2 executes while P1 stays WAITING. Waking P1 restores it once; a repeated wake is rejected.

Blocking, wake and state validation | actual C output

```
minios> block 1
OK: block 1
minios> step
[0,2) P2 Quiz         RUNNING -> FINISHED remaining=0
minios> tasks
Tasks | tick=2 quantum=2 records=2/20
PID NAME                 STATE     BURST LEFT ARRIVAL BYTES
1   Reading              WAITING   4     4    0       256
2   Quiz                 FINISHED  2     0    0       128
Ready queue: (empty)
minios> wake 1
OK: wake 1
minios> wake 1
ERROR: Only an existing WAITING task can be woken.
minios> run
[2,4) P1 Reading      RUNNING -> READY    remaining=2
[4,6) P1 Reading      RUNNING -> FINISHED remaining=0
Run paused: ready queue empty; WAITING tasks need wake.
```

Figure 2. Executed block/wake sequence from tests/resources.txt.

Memory budget, failure recovery and shutdown | actual C output

```
minios> task create FullBudget 1 65536
Created P1 FullBudget: READY, burst=1, payload=65536 bytes
minios> task create OverBudget 1 1
ERROR: Memory budget exceeded: requested=1 free=0 bytes.
minios> memory
Memory: used=65536/65536 bytes free=0 allocations=1
minios> run
[0,1) P1 FullBudget   RUNNING -> FINISHED remaining=0
Run paused: ready queue empty; WAITING tasks need wake.
minios> debug fail-next-allocation
Debug hook armed: next valid allocation will fail.
minios> task create ForcedFailure 1 64
ERROR: Forced allocation failure; no task was created.
minios> task create Survived 1 64
Created P2 Survived: READY, burst=1, payload=64 bytes
minios> check
CHECK PASS: queue, task states, and allocation ownership are consistent.
minios> memory
Memory: used=64/65536 bytes free=65472 allocations=1
minios> quit
Shutdown: used_bytes=0 allocations=0
```

Figure 3. Exact-budget allocation succeeds, one extra byte fails, completion reclaims memory, and an injected allocation failure leaves the next PID available. Shutdown clears the unfinished task's payload. Raw source: evidence/resources_console.txt.

Persistent notes and study sessions

Notes provide the study-room file service. A create writes the first line; append preserves it and adds the next line. A new MiniOS process reopens the saved text, proving persistence independently of task state.

Create, append and read a persistent note | actual C output

```
minios> note create revision CPU scheduling notes
OK: note create revision
minios> note append revision Round robin uses a FIFO queue.
OK: note append revision
minios> note read revision
CPU scheduling notes
Round robin uses a FIFO queue.
OK: note read revision
```

Figure 4. Note create, append and read in the first process.

A new process reopens and deletes the note | actual C output

```
minios> note read revision
CPU scheduling notes
Round robin uses a FIFO queue.
OK: note read revision
minios> note delete revision
OK: note delete revision
minios> note read revision
ERROR: Cannot read note: missing file or read failure.
minios> quit
Shutdown: used_bytes=0 allocations=0
```

Figure 5. A second process reads the same note, deletes it and reports the expected missing-file error. Its shutdown remains clean. Raw transcripts: notes_console.txt and notes_restart_console.txt.

File and session behavior

Service	Observed result and safeguards
Notes	C11 fopen, fgets, fputs, fclose and remove implement persistence. Existing notes cannot be overwritten by create; append requires an existing note.
File errors	Missing notes, invalid names, unavailable directories and a buffered write/close failure recover safely without claiming success.
Study sessions	A session is a scheduled task. A 5-tick session takes slices 2 + 2 + 1 and announces completion exactly once at tick 5 (T12).
Restart / reset	Notes persist. Tasks, PIDs and clock start fresh on process restart. Reset also releases all live task payloads.

Session completion and file outcomes are recorded in events.log with experiment, virtual tick, event, PID and result. These services execute sequentially and do not require real-time timers or background threads.

Verification and test results

Build environment: GCC 13.3.0 on Linux. The program compiles with -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror and zero warnings. Python 3.12 runs development tests; the application itself is C11.

Executed integration tests and C assertions | actual C output

```
T1 PASS | Reference scheduling and metrics
T2 PASS | Persistent note lifecycle
T3 PASS | Block/wake scheduling
T4 PASS | Empty, single, and short burst
T5 PASS | 20-task capacity
T6 PASS | Exact memory budget
T7 PASS | Input validation
T8 PASS | File failure recovery
T9 PASS | Invalid transitions and PIDs
T10 PASS | Forced allocation failure
T11 PASS | Memory lifecycle cleanup
T12 PASS | Study session completion
CORE ASSERTIONS PASS: exact metrics, blocked wait, late arrivals, rollback, capacity,
queue wrap, cleanup.
Result: 12/12 integration cases; core assertions PASS.
```

Figure 6. All 12 proposal test cases pass, followed by direct C assertions. Original output and observed outcomes are stored in evidence/test_console.txt and test_results.csv/json.

Test	Checked behavior	Result
T1	Exact trace, per-task metrics and means.	PASS
T2	Exact bytes survive restart; append/delete verified.	PASS
T3	Blocked PID excluded; wake is unique.	PASS
T4	Safe idle; short final slice uses remaining work.	PASS
T5	20 records accepted; record 21 rejected.	PASS
T6	Exact 65,536-byte budget and reclamation.	PASS
T7	11 invalid/oversized commands rejected.	PASS
T8	Read/open/path/write errors recover safely.	PASS
T9	Unknown PIDs and invalid transitions rejected.	PASS
T10	Forced allocation failure commits no partial task.	PASS
T11	Completion/reset/exit clear tracked payloads.	PASS
T12	One notice and event at session completion.	PASS

Additional checks: AddressSanitizer and UndefinedBehaviorSanitizer also pass T1-T12 and the C assertions, with leak detection disabled. LeakSanitizer cannot inspect /proc/PID/task in this environment, so no successful leak-tool result is claimed. Registry checks verify zero tracked payloads; full leak checking remains a team-machine action.

Challenges and remaining work

Implementation challenges and resolutions

Challenge	Resolution in the current prototype
Consistent state and queue	A shared model, bounded queue and post-command checks enforce one queue entry per READY task and none for WAITING/FINISHED.
Allocation failure and cleanup	Allocate before committing task/PID state. Use one ownership registry for completion, reset and shutdown; inject NULL failures for testing.
Correct waiting metrics	Count only time spent READY, rather than deriving waiting from turnaround when a task has been blocked.
Input and file error recovery	Parse bounded complete lines; discard oversized tails; reject invalid ranges and names; check buffered file close/write errors.
Environment-dependent verification	ASan/UBSan are available; LeakSanitizer is blocked by process-inspection restrictions. Preserve the diagnostic and rerun on the team host.

Plan to complete the remaining work

When	Remaining action	Completion evidence
5-6 Nov	Build on the agreed team machine; each member reviews their assigned module.	Local build/test outputs and reviewed interfaces.
7-8 Nov	Cross-review modules; rerun standard, sanitizer and full leak checks; resolve findings.	Actual local results and documented corrections.
9-10 Nov	Check names/IDs and report; verify ZIP; submit one group copy by 10 November.	Reviewed PDF plus source ZIP or GitHub link.
Final phase	Finalize design/limitations and actual contributions; rehearse the three proposal scenarios.	Team demonstration and final project documents.

Proposal ownership for the review

Yasmeen Alshehri: CLI and tasks. **Fatimah Alshehab:** memory. **Raneem Alshahrani:** queue, scheduling and metrics. **Sarah Almas:** notes, sessions, logs and evidence. These are the proposal's assignments; completed individual contributions and peer reviews must be confirmed by the team.

Current limits

The prototype models one CPU and virtual task execution. The 64 KiB limit measures task payloads only. Notes persist, while tasks do not. Writes are not transactional and may be partial when a file error occurs. Real-time reminders, a GUI and additional scheduling policies are outside the agreed core scope.